GoatVision: A Modular Computer-Vision Pipeline for Non-Contact Goat Monitoring

GoatVision Project Team

Technical report draft, October 2026

Abstract. Precision Livestock Farming (PLF) increasingly uses computer vision to monitor animals without handling them. This report describes GoatVision, a modular system that turns goat images or video into goat-level records through detection, segmentation-based annotation, ByteTrack identity tracking, view-quality assessment, deep-learning weight estimation and FAMACHA/conjunctiva anemia screening, served by a FastAPI backend and a web dashboard. We document the architecture, a green/red mask annotation that separates "goat detected" from "view usable for measurement", and the current weight model (ResNet-18, log-weight target, Huber loss). We state the limitations plainly: the weight model was trained on a 72-animal cattle dataset, no independent animal-level test has been run, and no validated goat-specific body condition score (BCS) model exists. GoatVision is therefore presented as a working engineering platform whose accuracy claims are still pending, and we specify the protocol needed to establish them.

Keywords: precision livestock farming, computer vision, goat monitoring, ByteTrack, weight estimation, body condition score, FAMACHA

1 Introduction

Weighing and inspecting goats by hand is labour-intensive and stressful for the animals. PLF addresses this with non-contact sensing, and computer vision is increasingly applied to animal identification, phenotyping, health monitoring, body-condition assessment and weight estimation. Open challenges include posture variation, lighting, occlusion, scarce labelled data and weak generalisation between farms.

Vision-based weight estimation has typically relied on body measurements such as body area and length, withers and hip height, hip width, body volume and chest girth, fed to regression models. Reported difficulties include posture changes, lighting, occlusion, environmental interference, a shortage of high-quality labelled datasets and poor transfer between farms, so a system should report its limits as carefully as its predictions.

GoatVision treats each goat as an identifiable entity rather than an unrelated image, so that

GoatVision treats each goat as an identifiable entity rather than an unrelated image, so that repeated observations accumulate into an individual record. Our contributions are:

- a modular, replaceable pipeline from raw image or video to a goat-level profile;
- a mask-filled green/red annotation that encodes measurement readiness;
- an identity-aware, multi-view design in which each goat accumulates evidence across frames;
- a candid audit of what is validated and what is not, with the protocol that closes the gaps.

2 System architecture

GoatVision is a FastAPI service with health and analysis endpoints and local media serving, consumed by a Next.js/React dashboard. Backend modules cover detection (`GoatDetector`), tracking (`GoatTracker` and a separate YOLO and ByteTrack video system), view evaluation (`BestViewEvaluator`), measurement extraction, weight regression, confidence calculation, visualisation, data logging and video processing. Figure 1 shows the intended flow, in which a view-quality gate sends unusable views to a warning instead of a measurement.

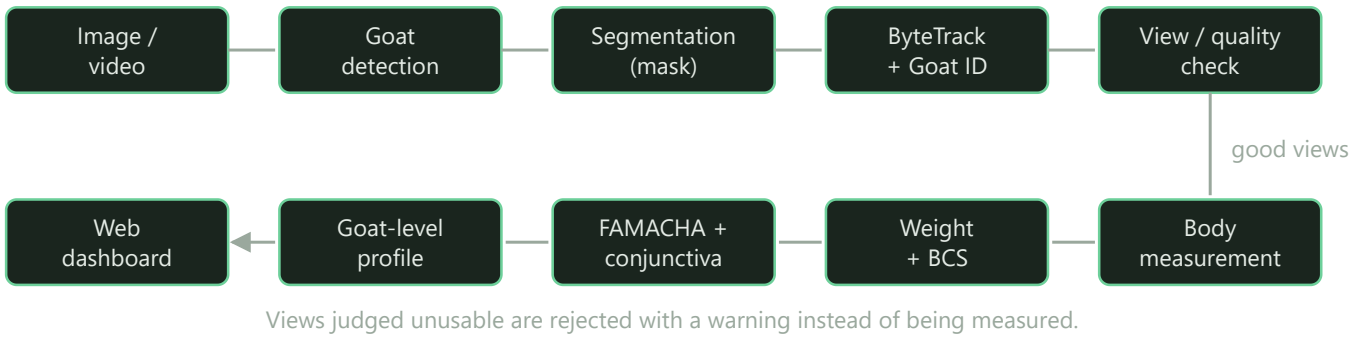


Figure 1. Intended GoatVision inference flow.

Two supporting services matter. A Roboflow workflow performs conjunctiva detection, with its API key held on the backend and never sent to the frontend. The `/api/health` endpoint reports whether the weight model is loaded and whether inference runs on CPU or CUDA, because GPU availability must not be assumed.

3 Methods

3.1 Detection and annotation

Goats are located by a detector and delineated by segmentation. The production overlay

fills the entire goat mask, green when the view is measurement-ready and red when it is not, and draws only the numeric goat ID at the mask centroid (Figure 2). Bounding boxes, class names, confidence, weight and BCS text are deliberately omitted; those values belong in the result panel. The separation matters because detection and measurement quality are different questions: a goat can be clearly detected yet photographed from an angle that makes any body measurement unreliable.



Figure 2. Schematic of the annotation design (illustration only, not model output). The whole mask is filled and only the numeric goat ID is drawn.

Figure 3. Annotated outputs and dashboard screenshots from GoatVision.

3.2 Tracking, identity and view quality

For video, a YOLO detector [3] feeds ByteTrack [2] so that a goat keeps one ID across frames, including brief occlusion or overlap. The tunable parameters are detection confidence, track-activation threshold, lost-track buffer, matching threshold and minimum consecutive frames. The system writes an annotated MP4, a frame-level CSV and the best measurement frame for each ID. `BestViewEvaluator` and a side classifier (`classify_side`) judge whether a frame shows a usable orientation, since measurements depend on visible body geometry; front views, partial bodies, occlusion and poor angles are rejected. Multi-view fusion (Figure 4) is designed so that a final result is aggregated over several good views of the same goat rather than trusting one photograph. An earlier Streamlit prototype already offered a weight-analysis tab and a ByteTrack tab that exported an annotated video, CSV and Excel logs, best frames and goat IDs, and it remains the functional reference for the web interface.



Figure 4. Multi-view fusion design. Production integration and validation are still pending.

3.3 Weight estimation

Two generations exist. The earlier morphology approach extracted body length and width, bounding-box size, mask area, elongation and compactness and fed them to Random Forest, ExtraTrees, Gradient Boosting, HistGradientBoosting, Ridge and ElasticNet regressors; it has been superseded. The current model is an image-to-weight network: a pretrained ResNet-18 [1] takes a 224 × 224 RGB image and regresses log(weight in kg), which is exponentiated at inference. The log target reduces scale problems and is paired with a robust Huber (SmoothL1) loss [5] (Table 1).

Table 1. Documented training configuration of the current weight model.

Setting	Value
Backbone and input	Pretrained ResNet-18, 224 × 224 RGB
Target and loss	log(weight_kg), Huber / SmoothL1
Epochs, batch size	25, 8 (early stopping, patience 8)
Learning rate, weight decay	3e-4, 1e-4 (backbone frozen for the first 2 epochs)
Augmentation	Random crop, horizontal flip, random affine, colour jitter
Split	80/10/10, animal-level, seed 42
Artifact	`weight_img_ds1_seed42.pt`

Training data comprise 72 animals with 5 views each (360 images) and a label file of animal ID and weight. Importantly, this is a multi-view dairy-cow dataset, not a goat dataset. Weights span roughly 40 to 1300 kg and include three suspect records near 950, 1100 and 1300 kg, likely bulls or data-entry errors. Because the split is by animal, views of one animal never appear in both training and test data.

The current network supersedes an older raw-kilogram model trained with mean-squared error. Training also uses an image decode and thumbnail cache so that the original high-resolution photographs are not repeatedly decoded.

3.4 Body condition score

BCS is planned on the usual 1 (very poor) to 5 (very fat) scale, but no validated goat model exists. The current estimator is a stub that returns 3.0 with confidence 0.0 and source "demo" unless a real artifact is present, so its output must not be presented as a prediction. Veterinary-grade labels cannot be created in-house, so a legitimate, pre-labelled goat BCS dataset is required; the current dataset contains no BCS ground truth.

3.5 Anemia screening

The FAMACHA service [4] (`/api/anemia/analyze`) assesses anemia from an uploaded

image. Separately, a Roboflow workflow localises the conjunctiva. The conjunctiva step is supplementary: if it fails, the FAMACHA result still stands. The output is a visual screening aid, not a veterinary diagnosis.

3.6 Interfaces and goat-level output

The backend exposes these endpoints:

- `GET /` and `GET /api/health` for service status, model status and CPU/CUDA mode;
- `GET /api/anemia/status` and `POST /api/anemia/analyze` for FAMACHA screening;
- `GET /api/anemia/conjunctiva/status` for the Roboflow integration (configured state, endpoint and provider, never the key).

The dashboard handles image and video upload, shows progress while the system is detecting, tracking and analysing, displays the green/red annotation and presents results in a side panel. Live camera input and per-goat history (previous and current weight and BCS, health trend) are planned. The target record for each goat contains its ID, estimated weight and confidence, BCS and confidence, anemia and conjunctiva findings, view quality and measurement readiness, and tracking details (frames seen, best frame). A value is shown only when the model behind it is genuinely available and validated.

4 Results and current status

4.1 Weight metrics

No final accuracy is claimed. Table 2 lists the recorded metrics of the earlier morphology model (V3). They describe a superseded approach, an R^2 near zero means it did little better than predicting the mean, and training metrics are not real-world accuracy. A previous audit also recorded a test MAE of about 178.7 kg, MAPE of 51.8% and R^2 of −0.165 before the data-quality problem was resolved. Neither set should be quoted as GoatVision's accuracy.

Table 2. Historical metrics, morphology model V3 (not final).

Metric	Cross-validation	Training
MAE	72.64 kg	66.48 kg
MAPE	15.48%	14.22%
RMSE	86.86 kg	80.23 kg
R^2	−0.0261	0.1245

4.2 Implementation status

Table 3. Component status.

Component	Status
Detection, mask annotation, goat IDs, ByteTrack, video processing, view and side classification	Developed
FastAPI backend, Roboflow conjunctiva integration	Developed and integrated
Web frontend	In development and integration
Weight model (ResNet-18)	Real model exists; independent accuracy pending
FAMACHA service	Exists; validation not reported
Multi-view fusion	Needs production integration
Goat-domain weight validation, independent weight benchmark, goat BCS model, labelled BCS data	Not available or pending

5 Planned validation protocol

Before any accuracy statement, the deployed model should be evaluated without retraining: load `weight_img_ds1_seed42.pt`, rebuild the exact dataset and seed-42 split, predict all five views of each test animal and average them per animal. Report animal-level MAE, MAPE, RMSE, R^2, the share of animals within ±5% and ±10%, median and maximum absolute error, and the number of test animals. Animal-level figures are the headline metric, since five photographs of one animal are not five independent samples.

6 Limitations

- **Domain mismatch.** Training data are cattle while the target is goats, so goat weight accuracy is unproven.
- **Small, noisy data.** Seventy-two animals cannot support strong generalisation claims, and three outlier labels distort regression.
- **No BCS ground truth.** There is no validated BCS model and no labelled BCS data.
- **Licensing.** The JXcow dataset is CC BY-NC-SA 4.0 and unsuitable for a commercial model; the Girolando image corpus was not available in usable form.
- **Field conditions.** Lighting, pose, occlusion, distance and camera angle are untested, and generalisation to a new farm has not been shown.
- **Fusion.** Multi-view fusion still needs explicit production integration and validation.

Until these are resolved, we recommend describing the system as providing computer

Until these are resolved, we recommend describing the system as providing computer-vision-based weight estimation undergoing independent and goat-domain validation, and automated visual screening support for anemia assessment, rather than as an accurate weight predictor or a diagnostic tool.

7 Discussion and roadmap

The main strengths are architectural: non-contact monitoring that avoids weighing every animal, individual tracking so that goats are distinguished rather than merely counted, video support that provides repeated observations, mask-based annotation, a combined weight, BCS and anemia view of each animal, and a modular FastAPI backend in which any model can be replaced without rebuilding the application. The recommended order of work is:

1. Finish and verify goat detection.
2. Finalise the green/red segmentation annotation.
3. Verify goat IDs and ByteTrack.
4. Audit the current weight model and run the independent test.
5. Establish the actual weight accuracy.
6. Find a legitimate, pre-labelled goat BCS dataset, then train and validate a BCS model and connect it to the backend.
7. Finalise FAMACHA and conjunctiva analysis.
8. Complete multi-view fusion and the goat-level dashboard.
9. Run end-to-end tests.

8 Conclusion

GoatVision has a working foundation: detection, mask annotation, persistent IDs, ByteTrack, view assessment, a weight network, FAMACHA and conjunctiva services, and a FastAPI and web interface. Its two scientific gaps are goat-domain weight validation and a genuine BCS model trained on legitimately labelled data. Section 7 sets out the order of work.

References

1. He, K., Zhang, X., Ren, S., Sun, J. (2016). Deep residual learning for image recognition. *Proc. IEEE CVPR*, 770–778.
2. Zhang, Y. et al. (2022). ByteTrack: Multi-object tracking by associating every detection box. *ECCV 2022*.

3. Redmon, J., Divvala, S., Girshick, R., Farhadi, A. (2016). You only look once: Unified, real-time object detection. *Proc. IEEE CVPR*, 779–788.
4. Van Wyk, J.A., Bath, G.F. (2002). The FAMACHA system for managing haemonchosis in sheep and goats by clinically identifying individual animals for treatment. *Veterinary Research*, 33, 509–529.
5. Huber, P.J. (1964). Robust estimation of a location parameter. *Annals of Mathematical Statistics*, 35(1), 73–101.